



Cómo viaja la información en el frontend

Vue 3 + Quasar + Vue Router + Pinia. El recorrido de un dato desde que el usuario lo escribe hasta que vuelve pintado en pantalla.

Alcance: este documento cubre **solo el frontend**. Donde aparece el servidor se dibuja como una nube gris, sin entrar en cómo funciona por dentro. Lo único que hay que saber de él es que recibe JSON y responde JSON.

Proyecto **estructura_frontend** · material de clase

El mapa de capas

Quién habla con quién. Ninguna capa se salta a la siguiente.

LO QUE VE EL USUARIO

`/views + /components + /layouts`

La vista es la dueña de la pantalla. Los componentes se dejan usar por cualquiera; el layout pone la barra y el menú.



llama a una función

LA CAPA QUE SABE HABLAR CON EL SERVIDOR

`/services/api.service.js`

Cuatro funciones sueltas: get, post, put, del. Reciben la URL y devuelven el dato.



usa la instancia configurada

LA CONFIGURACIÓN, EN UN SOLO SITIO

`/plugins/axios.js`

Pone la URL base, agrega el token a cada petición y ordena los errores antes de devolverlos.



HTTP + JSON

FUERA DEL FRONTEND

Servidor

Recibe JSON, responde JSON. Aquí no nos importa cómo.

A UN LADO: MEMORIA

`/store`

Lo que comparten varias pantallas: la sesión y los datos generales de la app.

A UN LADO: NAVEGACIÓN

`/router`

Decide qué vista se pinta y quién tiene permiso de entrar.

A UN LADO: HERRAMIENTAS

`/utils · /composables`

Reglas de validación, formato de fechas, avisos y confirmaciones.

La regla de oro: ningún componente llama a axios directamente. Si mañana cambia la dirección del servidor, se corrige un archivo y ninguna pantalla se entera.

1. Al abrir la aplicación

Qué pasa entre que se carga la página y aparece la primera pantalla

1 El navegador carga una página vacía

`index.html`

Solo tiene un `<div id="app"></div>`. Todo lo demás lo pinta Vue.

2 Se arma la aplicación y se le instalan las piezas

`main.js`

Es el único archivo que junta todo. Si algo tiene que estar disponible en toda la app, se registra aquí.

```
app.use(pinia) // memoria compartida
app.use(router) // navegación
instalarQuasar(app) // interfaz
app.mount("#app")
```

3 Pinia recupera la sesión guardada

`store/Auth.js` → `localStorage`

El store de sesión está marcado como `persist`, así que al arrancar se rellena solo con lo que quedó guardado en el navegador. Por eso al recargar con F5 uno sigue adentro.

4 El router mira la dirección y decide

`router/index.js`

Antes de dejar entrar, revisa si la ruta pide sesión. Sin sesión, devuelve al login.

5 Se pinta el marco y dentro la pantalla

`App.vue` → `AdminLayout.vue` → la vista

`App.vue` está casi vacío a propósito: solo cede el paso. El layout pone la barra y el menú una sola vez, y las vistas se van cambiando dentro de él sin volver a montarlo.

6 La vista pide sus datos

`onMounted()`

Recién cuando ya está en pantalla. De ahí en adelante empieza el flujo de la página siguiente.

2. Traer datos y mostrarlos

Ejemplo: la lista de cursos. Es el flujo de *lectura*.

1 La vista prende el indicador de carga

`CursosView.vue`

```
cargando.value = true;
error.value = null;
```

Siempre son **tres estados**: cargando, error y datos. Una pantalla que solo contempla el caso feliz está incompleta.

2 Pide los datos por su nombre, sin saber cómo se piden

`services/api.service.js`

```
cursos.value = await get("/cursos");
```

La vista no sabe de axios, ni de cabeceras, ni de la dirección del servidor. Solo dice *qué* quiere.

3 Se completa la petición y sale al servidor

`plugins/axios.js`

Aquí se le pega la dirección base y se le agrega el token de la sesión. Ninguna vista tiene que acordarse de eso.

4 Vuelve la respuesta, ya desempacada

`services/api.service.js`

El servicio devuelve directamente el contenido, no el sobre completo que entrega axios. La vista recibe el arreglo listo para usar.

5 El dato entra a una caja reactiva

```
const cursos = ref([])
```

Este es el momento clave. Al asignar el valor, Vue se entera solo y vuelve a pintar lo que dependa de él. Nadie tiene que "refrescar" nada a mano.

6 El dato baja al componente por props

`TablaDatos.vue`

```
<TablaDatos :filas="cursos" :columnas="columnas" :cargando="cargando" />
```

La tabla no sabe qué está pintando: recibe filas y columnas y las muestra. Por eso la misma tabla sirve para cursos y para aprendices.

7 Se apaga el indicador, pase lo que pase

`finally`

Va en `finally` para que el spinner se apague igual si la petición falló. Si no, quedaría girando para siempre.

3. Guardar lo que el usuario escribe

Es el flujo de *escritura*, y va al revés: de la pantalla hacia afuera.

1 Cada tecla actualiza el objeto del formulario

`v-model`

```
<q-input v-model="formulario.codigo" />
```

El enlace va en los dos sentidos: si el código cambia el valor, la pantalla también se actualiza sola.

2 Al enviar, primero se revisan las reglas

`utils/reglas.js`

```
:rules="[requerido('El código'), minimo(3, 'El código')]"
```

Cada regla devuelve true o el texto del error. El formulario las ejecuta todas y **solo si todas pasan** continúa. Si alguna falla, marca el campo en rojo y no sale ninguna petición.

3 Se arma el paquete de datos

Se limpia lo que sobra y se convierte lo que haga falta. Lo que solo servía para validar en pantalla (como confirmar la contraseña) no se envía.

4 Sale por la misma capa de siempre

`post(url, datos) / put(url, datos)`

Crear y actualizar comparten el mismo formulario: la única diferencia es si se está editando algo o no.

5 Se le avisa al usuario

`composables/useNotificar.js`

El mensaje que aparece en verde es el que devolvió el servidor, no uno inventado en el frontend.

6 Se cierra el formulario y se vuelve a pedir la lista

Se pide de nuevo en vez de "arreglar" la tabla a mano. Es una petición más, pero garantiza que lo que se ve es de verdad lo que quedó guardado.

Ojo: validar en el navegador es por comodidad, para responderle rápido al usuario. El servidor *vuelve* a validar siempre, porque un navegador se puede manipular.

4. El recorrido de la sesión

Cómo se consigue el permiso de entrar y cómo viaja después

1 El usuario entra con su correo y su contraseña

`LoginView.vue`

Es la primera pantalla de la aplicación. Sin permiso no se puede ver nada más.

2 El servidor devuelve un permiso y los datos del usuario

Ese permiso es un texto largo que sirve de credencial: quien lo tenga puede pedir información.

3 Se guarda en la memoria compartida

`store/Auth.js`

Va al store y no a la vista, porque lo necesitan **tres sitios que no se conocen entre sí**: la capa que hace las peticiones, el router y la barra superior.

4 Y de ahí, solo, al almacenamiento del navegador

`pinia-plugin-persistedstate`

No hay una sola línea escrita para guardarlo: el complemento lo hace por su cuenta. Eso es lo que permite recargar la página sin perder la sesión.

5 A partir de aquí, viaja en cada petición

`plugins/axios.js`

Se agrega automáticamente a todas. Ninguna pantalla tiene que acordarse de adjuntarlo.

6 Y el router lo consulta antes de dejar pasar

`router/index.js`

Las rutas marcadas como privadas preguntan al store si hay sesión. Si no la hay, avisan y devuelven al login.

Dos candados, no uno. El del router solo evita mostrar pantallas vacías; es comodidad. El candado de verdad está en el servidor, que sin credencial no entrega ni un dato. Saltarse el primero no sirve de nada.

5. Cuando algo falla

Un error también viaja, y también tiene su recorrido

EL CAMINO DE UN ERROR

1 Se ordena antes de repartirlo

`plugins/axios.js`

Un fallo de red, uno del servidor y uno de validación llegan con formas distintas. Se convierten todos a la misma forma para que las pantallas no tengan que distinguirlos.

2 Si el permiso venció, se resuelve en un solo lugar

Se cierra la sesión y se devuelve al login automáticamente, sin repetir esa revisión en cada pantalla.

3 Lo demás lo muestra la pantalla

Porque solo ella sabe el contexto: si venía llenando un formulario, si la lista quedó vacía, o qué campo fue el que el servidor rechazó.

6. Dónde vive cada dato

Las dos preguntas que más se repiten al escribir una pantalla nueva

¿ESTE DATO DÓNDE LO GUARDO?

Si el dato...	Va en...
lo usa una sola pantalla	<code>ref()</code> dentro de la vista — la lista de cursos, el texto del buscador, si el formulario está abierto
lo comparten varias pantallas	<code>/store</code> — la sesión del usuario, el título de la app, si el menú está abierto
debe sobrevivir a un F5	<code>persist</code> en el store — hoy solo la sesión
se calcula a partir de otro	<code>computed()</code> — nunca se guarda aparte, se deriva

El error más común es mandar todo al store global. Si un dato solo lo usa una pantalla, se queda en la pantalla: así se sabe de un vistazo quién lo puede cambiar.

CÓMO SE PASAN LA INFORMACIÓN LOS COMPONENTES

Mecanismo	Dirección	Para qué
<code>props</code>	de la vista hacia el componente	Pasarle los datos que debe mostrar
<code>slots</code>	de la vista hacia adentro	Decidir qué contenido va en un hueco que el componente deja libre
<code>emits</code>	del componente hacia la vista	Avisar que pasó algo, para que decida quien manda
<code>store</code>	en todas direcciones	Cuando los que comparten el dato no tienen relación de padre e hijo

Un componente que recibe todo por props y no consulta nada por su cuenta se puede reutilizar en cualquier pantalla. Uno que va a buscar sus propios datos, ya no: pertenece a una sola.